Take-Home Assignment: Bioprocess Scheduling and Closed-Loop Control

Role: Full-Stack Software Engineer, Machine Learning Focus

Effort: a few hours. Due within 5 days of receipt.

AI tools: Allowed and expected. Use them freely. If you use any, include the prompts you used in a `PROMPTS.md` file (see section 4). We evaluate your judgment, not your typing.

1. Context

Boston Bioprocess plans and executes manufacturing batches to develop products. A single batch is a long sequence of technical, interdependent tasks called unit operations: Seed, Bioreactor, TFF, Spray and Sum. Each unit operation runs on a specific, high-value piece of equipment and needs a person to run it. The available equipment is 1.5L, 15L, 20L, 75L and 1500L.

Today our teams coordinate batches, unit operations and equipment in spreadsheets like the one below. The process is slow, prone to error such as double-booking a piece of equipment, and cannot react quickly to change. A single scheduling error can lose a batch, set development back by weeks and cost significant capital.

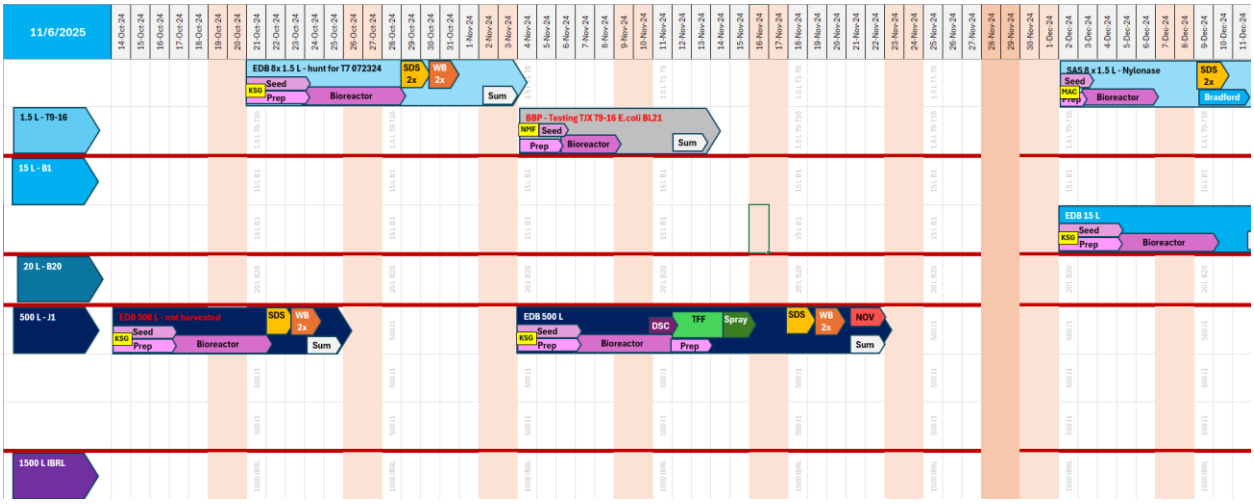


Figure 1. How batches are planned today: unit operations stacked on equipment lanes in a spreadsheet.

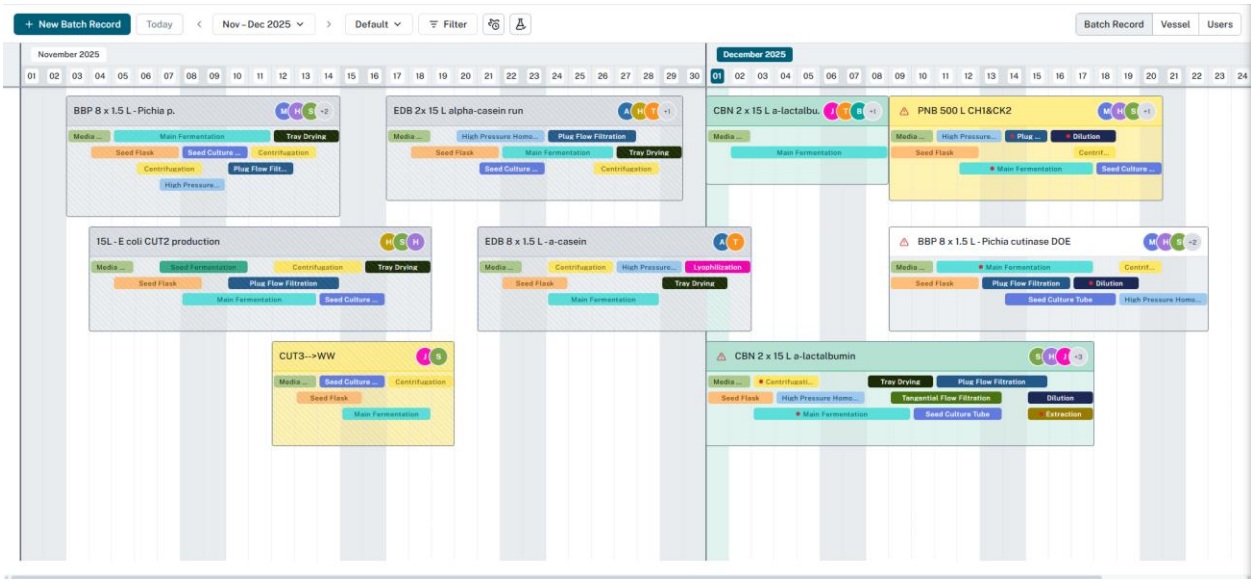


Figure 2. How a mature UI looks like (not expected but good to have some features)

Once a batch runs, an operator watches the bioreactor's dissolved oxygen (DO) and adjusts the feed pump by hand. Too much feed crashes DO and the cells starve. We are building a real-time control loop for this: a device pushes readings, a backend stores them, a model forecasts where DO is heading, a controller sends a feed setpoint back, and a live page shows all of it.

Your mission is a proof of concept that covers both: a web-based scheduling tool that enforces our business rules and is a single source of truth, and a small closed control loop for one bioreactor.

2. Your task

Build a desktop web application in three parts. Use any tools you prefer. We recommend React for the UI, FastAPI (Python) for the backend and PostgreSQL for the database, with SQLModel and Alembic if you use that stack, but SQLite is acceptable and so is any other stack. For the Gantt grid you may use any open-source scheduler library that supports resource timelines, or build a custom grid with CSS Grid or Flexbox. You are not expected to use a paid library. The evaluation focuses on functionality and code quality.

Part A. Scheduling backend: data model, API and business rules

Entities to model. Use these as a starting point; extend them as you see fit.

- `Equipment` (id, name). Seed with 1.5L, 15L, 20L, 75L, 1500L.
- `Batch` (id, name, start_date, end_date).
- `UnitOperation` (id, name, type, color, status, start_date, end_date, batch_id, equipment_id). `type` is one of Seed, Bioreactor, TFF, Spray, Sum. `status` is one of `draft`, `confirmed`, `completed`.
- `UnitOperationDependency` (id, from_unitop_id, to_unitop_id): an explicit "must finish before" link between two unit operations.

Business rules the backend must enforce. Treat end dates as exclusive: an operation that ends on November 4 frees its equipment on November 4.

1. A batch has a start and an end date. Every unit operation of the batch lies within them.
2. Unit operations follow the sequence Seed, Bioreactor, TFF, Spray, Sum. A batch need not include every type, but a later type may not start before every earlier type in the same batch has ended. Example: a batch with Seed on 1.5L from Oct 21 to Oct 24 may start its Bioreactor on 15L on Oct 24, not on Oct 23.
3. The same equipment cannot be used by two unit operations at the same time, whatever batch they belong to. Example: a Bioreactor on 1.5L from Oct 24 to Oct 31 and a Seed on 1.5L from Oct 29 to Nov 4 is a double-booking.
4. An explicit dependency's target may not start before its source has ended.
5. A `completed` unit operation cannot be moved or deleted.

Report each violation with a readable message that names the operations involved. Report each violating pair once per rule.

Behavior on save. Saving a unit operation that violates a rule must not silently succeed: either reject the save with the violation messages, or save it and return the violations so the UI can highlight the operation. Say in the README which you chose and why.

API. A minimal backend that persists to a database and exposes at least:

```
GET    /api/schedule?start_date=...&end_date=...   all data needed for the grid in that range,
including current violations
POST   /api/unit_operations
PUT    /api/unit_operations/{op_id}
DELETE /api/unit_operations/{op_id}
```

Seed data. Load at least three batches across the equipment, with at least two deliberate rule violations so the highlighting in Part B can be seen immediately.

Part B. Scheduler UI: the Gantt grid

- A Gantt-style grid. The y axis shows equipment lanes. The x axis is a scrollable date range with controls to choose the visible start and end dates.
- Unit operations appear as color-coded blocks on their equipment lane. All unit operations of one batch are grouped visually into a batch envelope, as in Figure 1.
- Clicking a unit operation opens a modal or drawer to edit its start, end, equipment (from the predefined list) and status.
- Any unit operation the backend reports as violating a rule is visually highlighted, for example a red border or an alert symbol, with the message available on hover or in a list.
- Desktop only. How the page looks and feels beyond this is up to you.

Part C. Closed-loop bioreactor control (one vessel)

Five pieces that form one closed loop. **A loop that closes matters more than an accurate model.**

In the real process a control action takes effect over minutes and a run lasts hours. Simulate that: run process time faster than wall-clock so you never wait real minutes or hours. Keep process time and wall-clock time separate in your data model, and stamp readings, forecasts and commands with process time.

1. **Device simulator.** A process that emits DO, pH, temperature and feed rate once per second by replaying a fermentation run (section 3). It accepts a feed-rate command and applies it: the feed signal follows the setpoint and DO responds with a lag, using the response model in section 3.
2. **Ingest API and store.** Receives pushed readings and writes them to a database. Tables for readings, model predictions and control commands at minimum. Predictions are stored with the process time they were made at and are never overwritten.
3. **Forecaster.** A small model (GRU, LSTM, linear, gradient boosting, or whatever you think is proper) that predicts the next 10 minutes of DO at one-minute resolution from the last 60 minutes of DO and feed rate. Report the error on held-out data next to the persistence baseline (repeat the last value), and say how you split.
4. **Controller.** A rule or a model-informed policy that emits a feed setpoint. Rule-based is acceptable if justified. Respect the bounds in section 3, the 5-minute actuator lag, and a dwell time between commands.
5. **Live page.** One page showing the DO and feed traces, the latest forecast overlaid, and every command marked on the time axis, with an input to send a manual setpoint. Polling every few seconds is enough.

Command validation. The backend must validate every command, from the controller or a person, and reject with a reason when the value is outside the bounds, the apply time is earlier than the device's current process time, the unit does not match, or another command for the same setpoint is still pending. Store rejected commands with their reason and show them on the live page.

3. Data and contracts

Runs. We provide three anonymized fermentation runs, attached to this assignment, as `run_A.csv`, `run_B.csv`, `run_C.csv`: about 40 hours each, one row per process-minute, four columns.

```
time_h,signal_name,value,unit
0.000,DO,98.4,%
0.000,pH,6.52,-
0.000,temp,30.0,degC
0.000,feed_rate,0.0,mL/h
```

Signals: `DO` (%), `pH` (-), `temp` (degC), `feed_rate` (mL/h). If you prefer, synthesize your own runs in the same shape: DO starts near 100%, falls as the culture grows, and drops further when the feed rate rises. Say in the README what you used.

Your simulator replays at a configurable speed. A sensible default is one process-minute per wall-clock second, so a command's 5-minute lag is visible within seconds and a 40-hour run replays in 40 minutes. For your recording you may run faster still, or start the replay partway into the run.

Limits. The feed setpoint bounds:

```
{"feed_rate": {"unit": "mL/h", "min": 0.0, "max": 30.0}}
```

Ingest and command schemas. Use these so your pieces stay comparable to ours; extend them if you need to.

```
POST /ingest
{"readings": [{"time_h": 12.350, "signal_name": "DO", "value": 38.2, "unit": "%"}, ...]}

POST /command
{"setpoint_name": "feed_rate", "value": 8.0, "unit": "mL/h", "apply_at_time_h": 12.400}
```

Device response to a command applied at process time `t0`:

- `feed_rate(t) = setpoint` for `t >= t0`, until the next command.
- `DO(t) = DO_run(t) + dDO(t)`, where dDO follows a first-order lag toward `-g * (feed_rate(t) - feed_run(t))` with time constant `tau = 5` process-minutes and gain `g = 2.0` %DO per mL/h. Clamp DO to `[0, 100]`.

Implement this as a simple step each second. Document any deviation.

4. Deliverables and submission

1. **Code** in a private GitHub repository named `<your_name>_bbp`. Invite `VinitBB` and FanLu55 (vkhatavkar@bostonbioprocess.com, flu@bostonbioprocess.com).
2. **A hosted link** to the running application, so that non-technical users can evaluate it. Hosting is required; Docker is not.
3. **A screen recording of at most three minutes,** committed to the repository or linked from the README, showing: the Gantt grid with violations highlighted, an edit in the modal that clears a violation, and the control loop closing: readings arriving, a forecast drawn, a command sent, the device responding.
4. **README.md** with how to run it locally, the hosted URL, and three headings: *How I split and evaluated the forecaster*, *Decisions and trade-offs* (at least two, from backend, frontend or ML), *How I would deploy, retrain and monitor this*. Include the seed data and where the deliberate violations are.
5. **PROMPTS.md** if you used any AI tool: which tools and models; the prompts you gave them, verbatim and in order (a coding-agent session export is fine); what you changed in their output; what they got wrong. Using AI is never penalized. Leaving this out when AI was used is.

5. Optional extensions

Do the base tasks first. Pick any extension you find interesting and mark it clearly in the README.

E1. AI scheduling assistant

An assistant that lets users change the schedule in natural language, for example "schedule a batch starting next Monday on the 15L reactor if possible, make sure Seed happens before Bioreactor and avoid equipment conflicts." The goal is to show you can integrate an LLM into production software safely, not just call an API.

- **Natural-language interface.** A chat box or command input on the scheduler page that accepts free-form requests about the schedule.
- **Tool-driven execution.** The assistant must call your backend to fetch the schedule, list equipment, and validate a proposed change with your Part A rules. It must not decide from its own knowledge alone.
- **Structured proposal.** The assistant turns the request into a structured set of creates, updates and deletes, and shows it to the user with the validation result.
- **Human in the loop.** Nothing is applied automatically. The user approves, edits or rejects the proposal; only an approved proposal is written through your API.

- **Deployment and serving, design only.** A short description of how this would be deployed, how a request flows through the system, and how you would handle latency, failures, logging and monitoring.
- **Deliverables.** The code, the tool definitions and orchestration logic, a README addendum covering prompt design, tool-calling strategy, validation approach, the human-in-the-loop workflow and the deployment design, and two or three example inputs with the system's outputs.

E2. Forecast-aware controller

Have the controller act on the forecast rather than only on the latest reading.

- Decide from where DO will be within one actuator lag (about 5 minutes ahead), not where it is now.
- Reason explicitly about the forecast horizon, the actuator lag, the dwell time and the setpoint bounds; say what happens when the forecast is stale or missing.
- Show one case in your recording or README where the forecast-aware policy acts earlier than the reactive rule would have, and what that did to DO.

E3. Drag to reschedule

On the Gantt grid, drag a unit operation to another day or another equipment lane. Save through your update endpoint and show the returned violations inline without a page reload. Describe how you keep the grid and the violation list consistent while a drag is in progress.

E4. Multiple vessels

Run two or more simulated vessels at once, each with its own loop, and show them on the live page. Describe what changed in the ingest path, the store and the controller to support more than one device, and where the design would break first at fifty vessels.